🏛 Structure — Defining, Declaring, Accessing & Initializing

📖 Definition

A **structure** is a user-defined data type that groups variables of **different types** under a single name, so related data can be treated and passed around as one unit.

❓ Why Do We Need It?

An array can only hold values of **one** type (all `int`, or all `float`...). But real-world data is rarely that uniform — a student record needs a name (text), age (integer), and marks (decimal) all bundled together. A structure lets you design exactly that kind of custom, mixed-type record.

🔄 Defining vs Declaring vs Initializing

```
struct Student {              // DEFINE — the blueprint, no memory used yet
    char name[20];
    int age;
    float marks;
};

struct Student s1, s2;        // DECLARE — creates actual variables (memory allocated)

struct Student s3 = {"Amit", 20, 89.5};   // INITIALIZE — values assigned in member order
```

Action	What It Does
Define	Describes the structure's shape (member names & types) — like designing a blank form
Declare	Creates an actual variable of that structure type — like printing a copy of the form
Initialize	Fills in starting values for the members — like filling out the form

🎯 Accessing Members — the Dot Operator

Once a structure variable exists, its individual members are accessed using the **dot operator** `.`

```
s1.age = 20;
strcpy(s1.name, "Amit");
s1.marks = 91.0;
```

💻 Example Program

```
#include <stdio.h>
#include <string.h>

struct Student {
    char name[20];
    int age;
    float marks;
};

int main() {
    struct Student s1;                // declare
    strcpy(s1.name, "Amit");          // access + assign
    s1.age = 20;
    s1.marks = 89.5;

    struct Student s2 = {"Riya", 21, 92.0};  // declare + initialize together

    printf("%s is %d, scored %.1f\n", s1.name, s1.age, s1.marks);
    printf("%s is %d, scored %.1f\n", s2.name, s2.age, s2.marks);
    return 0;
}
```

▶ Output

```
Amit is 20, scored 89.5
Riya is 21, scored 92.0
```

📝 Note

- The `struct Student` keyword combo must be repeated every time you declare a variable in C (unlike C++) — unless you use `typedef` to create a shorter alias.
- Initializer values must follow the **same order** the members were defined in, unless you use C99 designated initializers like `{.age = 21, .name = "Riya"}`.

✨ Fun Tip

A structure is a form template 📋 — defining designs the blank form, declaring prints a copy, and initializing fills it in!

🧩 Nested & Self-Referential Structures

📖 Nested Structure — Definition

A **nested structure** is a structure that contains **another structure** as one of its members — useful when a record naturally has a sub-group of related fields, like an address inside a student's record.

⚙️ Syntax & Access

```
struct Address {
    char city[20];
    char pin[10];
};

struct Student {
    char name[20];
    struct Address addr;      // nested structure member
};
```

Members inside the nested structure are reached by **chaining** the dot operator:

```
struct Student s1;
strcpy(s1.addr.city, "Pune");   // outer.inner.member
```

💻 Example Program — Nested Structure

```
#include <stdio.h>
#include <string.h>

struct Address { char city[20]; };
struct Student { char name[20]; struct Address addr; };

int main() {
    struct Student s1 = {"Amit", {"Pune"}};   // nested initializer
    printf("%s lives in %s\n", s1.name, s1.addr.city);
    return 0;
}
```

▶ Output

```
Amit lives in Pune
```

📖 Self-Referential Structure — Definition

A **self-referential structure** contains a **pointer to its own type** as a member. This is the building block behind linked lists, trees, and other dynamic data structures.

```
struct Node {
    int data;
    struct Node *next;   // pointer to another Node — NOT a Node itself
};
```

⚠ A structure can **never** contain a member of its **own type directly** — only a **pointer** to its own type. A direct member would need infinite memory to store itself; a pointer always has a fixed, known size regardless of what it points to.

💻 Example Program — A Tiny 2-Node Linked List

```
#include <stdio.h>

struct Node { int data; struct Node *next; };

int main() {
    struct Node n2 = {20, NULL};       // second node, points to nothing
    struct Node n1 = {10, &n2};        // first node, points to n2

    struct Node *cur = &n1;
    while (cur != NULL) {
        printf("%d -> ", cur->data);
        cur = cur->next;                // follow the pointer to the next node
    }
    printf("NULL\n");
    return 0;
}
```

▶ Output

```
10 -> 20 -> NULL
```

✨ **Fun Tip**

Nested structure is a box inside a box 📦. Self-referential structure is a treasure hunt map 🗺 — each clue (node) points to the location of the next one!

🎲 Bit-Fields

📖 Definition

A **bit-field** is a structure member that is given an **exact number of bits** instead of a whole byte or word — letting you pack several small values tightly together inside a single unit of memory. Commonly used for flags, status registers, and hardware-level programming.

🧩 Syntax

```
struct Flags {
    unsigned int isReady  : 1;   // uses only 1 bit  (values: 0 or 1)
    unsigned int hasError : 1;   // uses only 1 bit
    unsigned int mode     : 3;   // uses only 3 bits (values: 0 to 7)
};
```

The `: N` after each member name fixes exactly how many bits it may use — the compiler packs all of them together as tightly as it can, often into a single `int`-sized slot.

💻 Example Program — Size Comparison

```
#include <stdio.h>

struct FlagsBitfield {
    unsigned int isReady  : 1;
    unsigned int hasError : 1;
    unsigned int mode     : 3;
};

struct FlagsNormal {
    unsigned int isReady;
    unsigned int hasError;
    unsigned int mode;
};

int main() {
    struct FlagsBitfield f = {1, 0, 5};
    printf("Bit-field struct size = %lu bytes\n", sizeof(f));
    printf("Normal struct size    = %lu bytes\n", sizeof(struct FlagsNormal));
    printf("isReady=%u hasError=%u mode=%u\n", f.isReady, f.hasError, f.mode);
    return 0;
}
```

▶ Output

```
Bit-field struct size = 4 bytes
Normal struct size    = 12 bytes
isReady=1 hasError=0 mode=5
```

⚠ The exact bit layout is **compiler and platform dependent**, so bit-fields aren't portable across different systems. You also **cannot** take the address (`&`) of a bit-field member, since it may not start on a byte boundary.

📝 Note

- Bit-fields trade a small amount of extra CPU work (packing/unpacking bits) for a large saving in memory — worthwhile when you have many tiny flags.
- The maximum value a bit-field can hold is `2^N - 1` for `N` bits (e.g. 3 bits → 0 to 7).

✨ Fun Tip

A normal struct member is a whole parking spot for one car 🚗. A bit-field is a tiny reserved corner just big enough for a bicycle 🚲 — perfect when you only need a yes/no or a small number!

🏫 Arrays of Structures

📖 Definition

Just like an array of `int` holds many integers, an **array of structures** holds many structure records of the same type — perfect for storing a list of similar records, like an entire class of students.

🧩 Syntax

```
struct Student {
    char name[20];
    int marks;
};

struct Student class[3];      // array of 3 Student structures
```

🎯 Accessing & Initializing

Combine array indexing `[i]` with the dot operator to reach a specific record's member:

```
class[0].marks = 88;             // access

struct Student class[3] = {      // initialize all at once
    {"Amit", 88},
    {"Riya", 92},
    {"Zoya", 79}
};
```

💻 Example Program

```
#include <stdio.h>

struct Student { char name[20]; int marks; };

int main() {
    struct Student class[3] = {
        {"Amit", 88},
        {"Riya", 92},
        {"Zoya", 79}
    };

    for (int i = 0; i < 3; i++) {
        printf("%s scored %d\n", class[i].name, class[i].marks);
    }
    return 0;
}
```

▶ Output

```
Amit scored 88
Riya scored 92
Zoya scored 79
```

📝 Note

- An array of structures is stored in **contiguous memory**, exactly like a normal array — each element is simply the size of one structure.
- Looping through it with a `for` loop and `class[i].member` is the standard pattern for processing many records — the same pattern you'll reuse for reading marksheets, employee lists, inventories, and more.

✨ Fun Tip

A single structure is one filled-out form 🗒. An array of structures is a whole filing cabinet drawer full of them, all indexed and ready to flip through 🗂!

🔄 Structures and Functions

📖 Definition

A structure can be **passed to** a function and **returned from** one, just like any ordinary variable. By default, this happens **by value** — exactly the same call-by-value behaviour you learned for plain variables.

⚖️ Passing a Structure by Value

When a structure is passed as a function argument, C copies the **entire structure** into the function's parameter. The function works only on this copy — any changes made inside it do **not** affect the original structure back in the caller.

💻 Example Program — Passing & Returning a Structure

```c
#include <stdio.h>

struct Student { char name[20]; int marks; };

void tryToChange(struct Student s) {     // receives a COPY
    s.marks = 100;                        // only changes the copy
    printf("Inside function: marks = %d\n", s.marks);
}

struct Student boostMarks(struct Student s) {  // returns a NEW structure
    s.marks += 5;
    return s;
}

int main() {
    struct Student s1 = {"Amit", 88};

    tryToChange(s1);
    printf("After tryToChange: marks = %d\n", s1.marks);   // unchanged!

    s1 = boostMarks(s1);                  // caller reassigns the returned copy
    printf("After boostMarks: marks = %d\n", s1.marks);
    return 0;
}
```

▶ Output

```
Inside function: marks = 100
After tryToChange: marks = 88
After boostMarks: marks = 93
```

⚠ Passing a large structure by value means the **whole thing gets copied** every single call — for structures with many or large members, this wastes both time and memory. The fix is to pass a **pointer** to the structure instead — covered next.

📝 Note

- To actually modify the caller's original structure, you must either return the modified copy and reassign it (as shown above), or pass a pointer to it (next page).
- A function can return a structure directly — the whole structure is copied out to the caller, same as with parameters.

✨ Fun Tip

Passing a structure by value is like handing someone a photocopy of an entire filled-out form 📋 — they can scribble all over it, but your original stays untouched!

👉 Structures and Pointers

📖 Definition

Just like any variable, you can create a **pointer to a structure** — storing the structure's address instead of copying the whole structure itself. This is the efficient way to work with structures, especially when passing them to functions.

⚙️ Syntax

```
struct Student s1 = {"Amit", 88};
struct Student *ptr = &s1;      // ptr holds the ADDRESS of s1
```

🎯 Accessing Members — the Arrow Operator

Through a pointer, members are accessed using the **arrow operator** `->` instead of the dot operator:

Form	Meaning
`ptr->marks`	Shorthand, most commonly used
`(*ptr).marks`	Equivalent long-hand form — dereference first, then access with `.`

⚠ The parentheses in `(*ptr).marks` are required — without them, `*ptr.marks` is read by C as `*(ptr.marks)`, which is a compile error since `.` binds tighter than `*`.

💻 Example Program — Modifying the Original via a Pointer

```
#include <stdio.h>

struct Student { char name[20]; int marks; };

void boostMarks(struct Student *ptr) {   // receives an ADDRESS, not a copy
    ptr->marks += 5;                      // modifies the ORIGINAL structure directly
}

int main() {
    struct Student s1 = {"Amit", 88};

    boostMarks(&s1);                     // pass the address
    printf("marks = %d\n", s1.marks);     // original IS changed this time
    return 0;
}
```

▶ Output

```
marks = 93
```

📝 Note

- Passing a pointer avoids copying the whole structure — only one address (usually 8 bytes) is copied, no matter how big the structure is.
- This is exactly the **call by reference** technique from the Pointers and Functions units, now applied to structures.

✨ Fun Tip

Dot (`.`) is for when you're holding the form itself. Arrow (`->`) is for when you only have directions to where the form is kept 🗺!

🔗 Union — Basics

📖 Definition

A **union** groups different-type variables under one name, just like a structure — but with one crucial difference: **all members share the exact same memory location**. Only **one** member can hold a valid value at any given moment.

🧩 Syntax — Looks Just Like a Structure

```
union Data {
    int i;
    float f;
    char str[20];
};

union Data d1;          // declare
d1.i = 10;               // access — same dot operator as structures
```

📊 Why the Size Is Different

A structure's size is the **sum** of all its members' sizes. A union's size is only as big as its **largest** member — because every member is overlaid on top of the same shared memory.

💻 Example Program — Shared Memory in Action

```
#include <stdio.h>

union Data { int i; float f; char str[20]; };
struct DataStruct { int i; float f; char str[20]; };

int main() {
    union Data d;
    printf("Union size  = %lu bytes\n", sizeof(d));
    printf("Struct size = %lu bytes\n", sizeof(struct DataStruct));

    d.i = 10;
    printf("After setting d.i = 10:  d.i = %d\n", d.i);

    d.f = 3.14;                          // OVERWRITES d.i — same memory!
    printf("After setting d.f = 3.14: d.i = %d (garbage now)\n", d.i);
    printf("                          d.f = %.2f (valid)\n", d.f);
    return 0;
}
```

▶ Output

```
Union size  = 20 bytes
Struct size = 28 bytes
After setting d.i = 10:  d.i = 10
After setting d.f = 3.14: d.i = 1078523331 (garbage now)
                          d.f = 3.14 (valid)
```

⚠ Writing to one union member **corrupts** any value previously stored in another member, since they occupy the same bytes. It's the programmer's responsibility to keep track of which member currently holds a meaningful value.

✨ Fun Tip

A structure is a house where every family member has their own room 🏠. A union is a single studio apartment 🏘 — whoever moves in last is the only one actually living there!

⚖ Structure vs Union & Structure Within Union

📖 Structure vs Union — Key Differences

Aspect	Structure	Union
Memory used	Sum of all members' sizes	Size of the **largest** member only
Member storage	Each member has its own separate space	All members **share** the same space
Valid members at once	All members hold valid values simultaneously	Only the **most recently written** member is valid
Keyword	`struct`	`union`
Typical use	Records with many fields needed together (student, employee)	One value at a time, of varying type (a variant/tagged value)

🏗 Structure Within a Union

A union's member can itself be a **structure** — this is common when one "variant" of the union needs to bundle several related fields together as a single option.

```
struct Point { int x, y; };

union Shape {
    float radius;         // option 1: a circle needs just a radius
    struct Point corner;   // option 2: a rectangle needs an x,y point (a whole struct!)
};
```

💻 Example Program — Structure Nested Inside a Union

```
#include <stdio.h>

struct Point { int x, y; };
union Shape { float radius; struct Point corner; };

int main() {
    union Shape s;

    s.radius = 5.0;                      // using it as a circle
    printf("As circle: radius = %.1f\n", s.radius);

    s.corner.x = 3;                      // switch to using it as a rectangle corner
    s.corner.y = 4;                      // this OVERWRITES s.radius — same memory
    printf("As rectangle corner: (%d, %d)\n", s.corner.x, s.corner.y);
    printf("radius is now garbage: %.1f\n", s.radius);
    return 0;
}
```

▶ Output

```
As circle: radius = 5.0
As rectangle corner: (3, 4)
radius is now garbage: 0.0
```

📝 Note

- This pattern — a union of several structures — is the classic building block for a **tagged union** (an extra "type" field outside the union tells you which member is currently valid).
- The reverse — a **structure containing a union** as one of its members — is just as common, e.g. a record where one field's meaning depends on a "type" flag stored elsewhere in the same structure.

✨ Fun Tip

A structure is a backpack with separate labeled pockets 🎒. A union is a magician's single hat 🎩 — it can hold a rabbit or a dove, but never both at the same time!

Compiled & Prepared By
Anand Sagar Rout
Bs.c Computer Science | GMU